

Transfer Learning Final

Nina Carrillo (1940991)

UCSC – Spring 2026

1 Introduction

1.1 Problem and Goal Setting

The objective of this project is to classify images into one of 100 categories using a highly limited training dataset containing approximately 10 labeled images per class, or roughly 1,000 images in total. The challenge is designed to evaluate the effectiveness of transfer learning in low-data scenarios, with the goal of surpassing the baseline Kaggle test accuracy of 60%. The dataset combines images from multiple sources, resulting in substantial variation in image content, resolution, aspect ratio, and visual appearance. Because the problem involves a large number of classes but only a small number of examples per class, training a deep neural network from scratch would be impractical and unlikely to generalize well.

1.2 Why transfer learning is appropriate.

Transfer learning is particularly well suited for this task because of the limited amount of labeled training data. Deep neural networks typically require large datasets to learn meaningful visual representations from scratch. With only a few examples available for each class, a model trained from random initialization would likely overfit the training set and perform poorly on unseen data. By leveraging a model pretrained on a large-scale dataset such as ImageNet, the network already possesses rich visual features that capture edges, textures, shapes, and object-level patterns. Fine-tuning a pretrained model allows these learned representations to be adapted to the target dataset while requiring significantly less data, training time, and computational resources.

1.3 Brief summary of your approach and main result.

I used a Vision Transformer (ViT-B/16) pretrained on ImageNet via TorchVision. The original classification head was replaced with a new head for 100 classes, and the model was fine-tuned using AdamW, cosine learning rate scheduling, MixUp/CutMix augmentation, and label smoothing. My best configuration fine-tuned the classification head plus the last transformer block and achieved strong training accuracy along with a competitive Kaggle submission score.

2 Dataset

2.1 Number of classes and dataset sizes (train/val/test).

Split	Approx. Size
Train	1,000 images (10 per class)
Validation	Optional split from training set
Test	1,000 images
Classes	100

Table 1: The training set contains approximately 10 labeled images per class, making the task data-scarce and well-suited for transfer learning.

2.2 Directory structure and label mapping details (ensure labels 0–99 match folders).

The dataset follows the standard `torchvision.datasets.ImageFolder` organization, where training images are stored as `train//.jpg`, with a separate folder corresponding to each class. Test images are stored in a single directory as `test/.jpg` and do not contain labels. A critical detail of this dataset is the mapping between class folders and numerical labels. `ImageFolder` assigns class indices based on the lexicographic ordering of folder names rather than their numerical values. As a result, folders are ordered as 0", 1", 10", 11", ..., 2", 20", and so on. Consequently, the model output index does not necessarily correspond directly to the integer value of the class

name. To ensure correct predictions during inference, the predicted class index is converted back to its original folder name using the `ImageFolder(...).classes` mapping. This guarantees that each prediction is associated with the proper class label and prevents errors caused by mismatched label ordering.

2.3 Preprocessing (resize, normalization) and data augmentation used.

All images were preprocessed to match the input requirements of the pretrained Vision Transformer model. During evaluation and inference, images were resized to 384×384 pixels using bicubic interpolation, converted to tensors, and normalized using the standard ImageNet mean values of $[0.485, 0.456, 0.406]$ and standard deviations of $[0.229, 0.224, 0.225]$. During training, several data augmentation techniques were applied to improve generalization and reduce overfitting. Images were randomly cropped and resized using `RandomResizedCrop` with a scale range of 0.7 to 1.0, randomly flipped horizontally, and subjected to color jittering to vary brightness, contrast, saturation, and hue. In addition, MixUp ($\alpha = 0.2$) and CutMix ($\alpha = 1.0$) were applied at the batch level to generate synthetic training examples and encourage the model to learn more robust decision boundaries. These augmentations effectively increased the diversity of the training data and improved performance in the low-data setting.

3 Implementation

3.1 Model

3.1.1 Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.

I used Vision Transformer ViT-B/16 from TorchVision with pre-trained ImageNet SWAG weights:

```
torchvision.models.vit_b_16(  
    weights=ViT_B_16_Weights.IMAGENET1K_SWAG_E2E_V1,  
    image_size=384  
)
```

ViT was chosen because transformer-based vision models perform well on diverse image classification tasks and benefit strongly from large-scale pretraining.

For this project, I used the Vision Transformer (ViT-B/16) architecture provided by TorchVision. Specifically, the model was initialized using pretrained ImageNet SWAG weights and configured to operate on 384×384 input images. A Vision Transformer was selected because transformer-based architectures have demonstrated strong performance on image classification tasks and benefit significantly from large-scale pretraining. By leveraging a model that had already learned rich visual representations from ImageNet, I was able to adapt it efficiently to my 100-class classification problem despite having a relatively small training dataset.

3.1.2 Architecture changes (classifier head, pooling, dropout, etc.).

The original classifier head was replaced with a dropout layer followed by a linear layer for 100 classes:

```
model.heads.head = nn.Sequential(  
    nn.Dropout(0.4),  
    nn.Linear(in_features, 100)  
)
```

A dropout rate of 0.4 was used to improve regularization.

To adapt the pretrained model to the task, I replaced the original classification head with a custom classifier designed for 100 output classes. The new head consisted of a dropout layer with a dropout probability of 0.4 followed by a fully connected linear layer. The addition of dropout served as a regularization mechanism to reduce overfitting by preventing the model from relying too heavily on specific neurons during training.

3.1.3 Fine-tuning strategy (frozen layers, unfreezing schedule).

We froze most of the backbone and fine-tuned only the classifier head and the last transformer block:

```
freeze_backbone(model, unfrozen_layers=1)
```

This strategy reduces the number of trainable parameters and helps prevent overfitting on the small dataset while still adapting high-level features to the new task.

Rather than fine-tuning the entire network, I employed a partial fine-tuning strategy. Initially, all parameters in the backbone were frozen, and only the classifier head remained trainable. I then unfroze the final transformer block, allowing the model to adapt higher-level features while preserving the majority of the pretrained representations. This approach significantly reduced the number of trainable parameters and improved training stability, which was especially important given the limited size of the dataset.

4 Training

4.0.1 Loss function and optimizer

The model was trained using the `CrossEntropyLoss` function with a label smoothing factor of 0.1. Label smoothing was incorporated to improve generalization by preventing the model from becoming overly confident in its predictions. Optimization was performed using the AdamW optimizer, which is commonly used for transformer architectures due to its effective handling of weight decay and adaptive learning rates.

4.0.2 Learning rate, scheduler, batch size, epochs, weight decay.

Training was conducted using a learning rate of 110^{-4} , a weight decay coefficient of 0.05, a batch size of 16, and a total of 30 training epochs. To improve convergence, I employed a `CosineAnnealingLR` learning rate scheduler that gradually reduced the learning rate throughout training. This scheduling strategy helps the optimizer make larger updates during the early stages of training while allowing for more refined adjustments later in the optimization process.

4.0.3 Hardware/software environment (GPU, PyTorch version)

The project was implemented using PyTorch and TorchVision and trained on Volga, a high-performance Linux computing server maintained by UC Santa Cruz. Volga is equipped with an Intel Xeon Gold processor featuring 32 cores (64 threads) running at 2.1 GHz and 256 GB of RAM, providing sufficient computational resources for fine-tuning large deep learning models. The system runs Red Hat Enterprise Linux 9.4 and includes substantial storage capacity for datasets and model checkpoints. Using Volga allowed training to be conducted in a stable, dedicated computing environment without

the hardware limitations of a personal laptop, enabling efficient experimentation with Vision Transformer architectures and data augmentation techniques.

5 Experiments

5.1 Baseline Setup

Our initial benchmark model consisted of a pretrained Vision Transformer (ViT-B/16) adapted for the 100-class classification task. To accomplish this, the original ImageNet classification layer was removed and replaced with a new fully connected layer containing 100 output neurons. During baseline training, only the newly added classification head was updated while all pretrained backbone parameters remained frozen. This configuration allowed us to evaluate how much performance could be obtained from the pretrained visual representations alone before introducing additional fine-tuning strategies.

5.2 Hyperparameter tuning plan and validation method.

To systematically evaluate model performance, the available labeled images were divided into training and validation subsets using an 80/20 split. The split was performed using stratified sampling so that each class maintained approximately the same proportion of samples in both subsets. This was particularly important because the dataset contained a limited number of examples per category, making it essential to preserve class balance during evaluation.

Throughout training, validation accuracy was monitored after each epoch, and the model checkpoint with the highest validation accuracy was retained. A series of experiments were conducted to identify an

effective combination of training parameters. The primary variables explored included learning rate, weight decay, dropout probability, number of training epochs, augmentation intensity, and the number of transformer layers allowed to update during fine-tuning. Each experiment was compared against previous configurations to determine whether the modification improved validation performance. Once an effective set of hyperparameters had been identified, the validation split was removed and the model was retrained using the entire labeled dataset so that all available examples could contribute to learning before generating the final competition submission.

5.3 Ablations (augmentations, model size, freezing strategy, LR, etc.).

Several ablation studies were performed to determine which design choices contributed most significantly to performance. One of the most impactful changes involved partially fine-tuning the Vision Transformer backbone rather than keeping it completely frozen. Allowing the final transformer block to update alongside the classification head produced substantially higher accuracy than training the classifier alone, indicating that modest adaptation of high-level features was beneficial for the target dataset.

We also evaluated the effect of advanced augmentation techniques. Incorporating MixUp and CutMix introduced additional variation into the training process by combining samples and labels within each batch. Compared with training on unmodified images, these augmentations improved generalization and produced a measurable increase in validation accuracy. Additional experiments focused on regularization and optimization settings. After testing several combinations, a learning rate of 110^{-4} together with dropout regularization yielded the most reliable performance and improved validation

results compared with earlier configurations.

Finally, after selecting the best-performing hyperparameters, the model was trained using the complete training dataset rather than reserving a validation subset. Because the dataset contained only a small number of examples per class, exposing the model to every available labeled image allowed it to learn more representative features. This final training strategy produced the strongest overall performance and led to the highest test-set accuracy among all configurations evaluated.

6 Results

6.1 Training/validation accuracy (and loss) and any key curves/plots.

6.2 Kaggle public leaderboard score (and submission settings).



Figure 1: Kaggle Leaderboard Placement

6.3 Brief qualitative examples or error analysis highlights (optional).

7 Discussion

The most successful aspects of the project were the use of pretrained Vision Transformer weights, partial fine-tuning of the backbone, and advanced augmentation techniques such as MixUp and CutMix. Together, these methods improved the model’s ability to generalize from a limited number of training examples while reducing the risk of overfitting. Label smoothing further contributed to training stability by discouraging overly confident predictions.

Despite these successes, several limitations remained. The relatively small size of the training dataset restricted the model’s exposure

to variations within each class, making it difficult to learn robust representations for all categories. Overfitting was observed when augmentation techniques were removed or when the model relied too heavily on a small subset of training examples. Additionally, some visually similar classes remained difficult to distinguish even after fine-tuning.

Future improvements could focus on unfreezing additional transformer blocks while reducing the learning rate to allow deeper adaptation of pretrained features. Alternative architectures such as larger Vision Transformer variants or EfficientNet models could also be explored. Furthermore, techniques such as test-time augmentation, model ensembling, and cross-validation could potentially yield additional performance gains.

8 Reproducibility

To improve reproducibility, a fixed random seed of 42 was used throughout the project. The seed was applied to Python’s random module, NumPy, and PyTorch to reduce variability between training runs. While complete determinism may depend on hardware and backend implementation details, fixing the seed helps ensure that experimental results remain reasonably consistent.

The project was implemented using Python 3.10 or newer together with PyTorch, TorchVision, Pandas, Pillow, and Matplotlib. All dependencies can be installed through standard package management tools such as pip. Training can be reproduced by executing the `run.py` script, which loads the dataset, initializes the model, and performs fine-tuning. After training, the `predict.py` script can be executed to generate `submission.csv` for leaderboard evaluation.

9 Team Contributions

Me. It was all me. I hated this final. I did this. All by myself. Not happy about it.